Carleton University

SYSC 3303A Project Report
Project: Firefighting Drone Swarm
Team A3-7

Team Member:
- Ma Chenwei (101315192)
- Shah Asaad (101173123)
- Ye Edmund (101289078)
- Zhang Derrick (101232374)

Table of Contents

1. Introduction ---------- 3
2. Diagrams and System Design ---------- 4
   2.1 UML Class Diagram ---------- 4
   2.2 State Machine Diagrams ---------- 5
      2.2.1 Drone State Machine ---------- 5
      2.2.2 Scheduler State Machine ---------- 7
   2.3 Sequence Diagrams ---------- 8
      2.3.1 Normal Fire Response ---------- 8
      2.3.2 DRONE_STUCK ---------- 10
      2.3.3 NOZZLE_JAM ---------- 12
   2.4 Timing Diagrams ---------- 14
3. Team Responsibilities ---------- 15
4. Setup and Test Instructions ---------- 17
   4.1 Prerequisites ---------- 17
   4.2 To Run the Project ---------- 17
   4.3 To Run Tests ---------- 17
5. Measurement Results ---------- 18
6. Design Reflection ---------- 20

1. Introduction

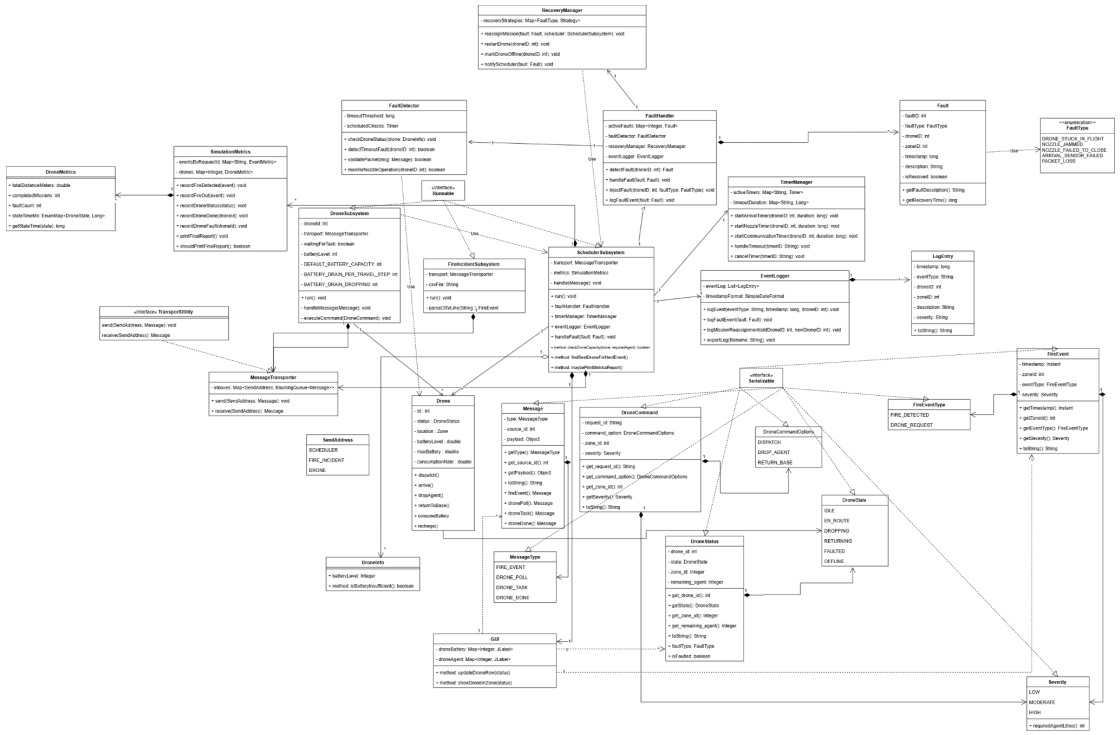
This project presents the design and implementation of a real-time distributed system for coordinating a firefighting drone swarm. The goal of the system is to efficiently respond to fire incidents by dispatching autonomous drones to different zones while minimizing response time and ensuring system reliability.

The system is composed of three main subsystems: the Scheduler, the Drone Subsystem, and the Fire Incident Subsystem. The Scheduler acts as the central controller, responsible for assigning tasks to drones and handling faults. The Drone Subsystem simulates the behavior and state transitions of individual drones, including movement, agent deployment, and refilling. The Fire Incident Subsystem generates fire events based on input files and monitors the status of each zone.

This project emphasizes real-time behavior, concurrency, and distributed communication. The subsystems communicate using UDP sockets and operate as independent processes. Additionally, the system includes a graphical user interface (GUI) to visualize drone activities, fire incidents, and fault conditions.

Through iterative development, the system evolves to support multiple drones, fault handling, capacity constraints, and performance measurement. The final implementation demonstrates the effectiveness of the scheduling strategy and the robustness of the system under various scenarios.

2. Diagrams and System Design
2.1 UML Class Diagram:



The system consists of three independent subsystems communicating via UDP sockets:

SchedulerSubsystem (Port 6000):

- Central controller managing the drone fleet
- Receives FireEvent messages from FireIncidentSubsystem
- Tracks DroneInfo for each drone (location, remaining agent, battery level)
- Implements scheduling logic via findBestDroneForNextEvent()
- Maintains SimulationMetrics for performance tracking
- Forwards messages to GUI for visualization

DroneSubsystem (Ports 6100-6120):

- Individual drone simulator
- Implements state machine (IDLE → EN_ROUTE → DROPPING → RETURNING → IDLE)
- Manages resources: remainingAgent (0-100L) and batteryLevel (0-100%)
- Tracks position (posX, posY) during flight
- Executes DroneCommand received from Scheduler
- Reports DroneStatus back to Scheduler with location and resource levels

FireIncidentSubsystem (Port 7000):

- Fire event generator
- Reads FireEvent from CSV input file with timestamp, zone, severity, fault type
- Sends FIRE_EVENT to Scheduler
- Receives FIRE_OUT confirmation
- Maintains outstandingFires counter per zone

GUI:

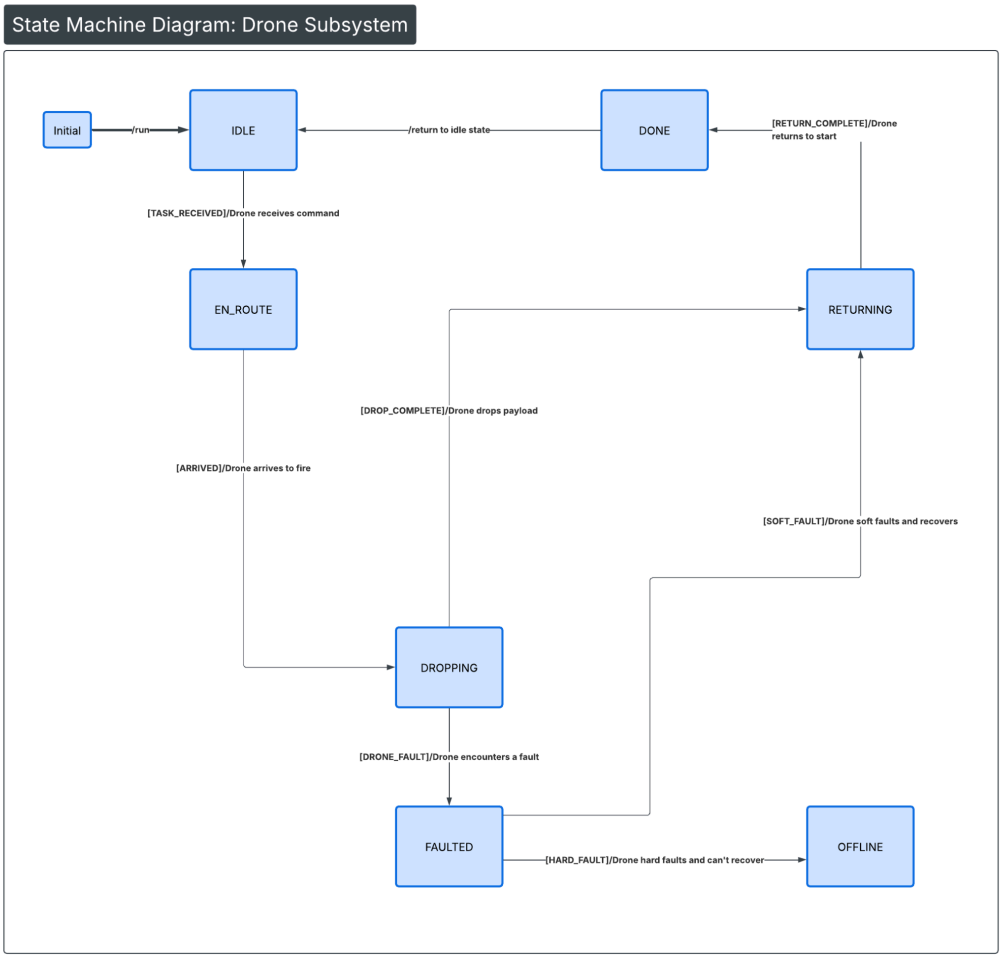
- Real-time visualization display
- Stores Map<Integer, DroneStatus> for all drones
- Stores Map<Integer, FireEvent> for all active zones
- Displays drone state with color coding (IDLE=Gray, EN_ROUTE=Blue, DROPPING=Orange, etc.)
- Shows battery level with color warnings (Red ≤20%, Orange 21-50%, Green >50%)
- Shows remaining agent with color warnings (Red ≤30L, Orange 31-60L, Green >60L)
- Updates in real-time via handleIncomingMessage()

Supporting Classes:

- Message: UDP packet format with type (FIRE_EVENT, DRONE_TASK, DRONE_STATUS, etc.)
- DroneInfo: Scheduler's metadata for each drone (droneId, position, capacity, state)
- DroneStatus: Real-time status report from drone to Scheduler
- DroneCommand: Task assignment from Scheduler to drone
- FireEvent: Fire incident specification (zone, severity, fault type)
- SimulationMetrics: Collects performance data (response time, distance, state times)

2.2 State Machine Diagrams

2.2.1 Drone State Machine:



Each drone follows an autonomous state machine with 6 states:

IDLE (Initial State)
- Drone waiting at base for tasks
- Transition to EN_ROUTE when receives DRONE_TASK command
- Event: TASK_RECEIVED

EN_ROUTE (Flying to Zone)
- Drone traveling toward assigned fire zone
- Battery drains at rate of 1% per travel step
- Monitors for DRONE_STUCK fault (timeout after configured delay)
- Two possible transitions:
- → DROPPING when arrival sensor detects zone reached (Event: ARRIVED)
- → FAULTED if DRONE_STUCK fault detected (Event: FAULT_DETECTED)
- → OFFLINE if NOZZLE_JAM detected at arrival (Event: HARD_FAULT)

DROPPING (Releasing Agent)
- Drone at zone, nozzle open, releasing firefighting agent
- Agent depletes based on severity (LOW=20L, MODERATE=50L, HIGH=100L)
- Battery drains at rate of 1% per 2-second interval
- Single transition:
- → RETURNING when agent deployment complete (Event: DROP_COMPLETE)

RETURNING (Flying Back to Base)
- Drone traveling back to base
- Battery continues draining at 1% per travel step
- Single transition:
- → IDLE when base reached, auto-refill to 100L agent and 100% battery (Event: RETURN_COMPLETE)

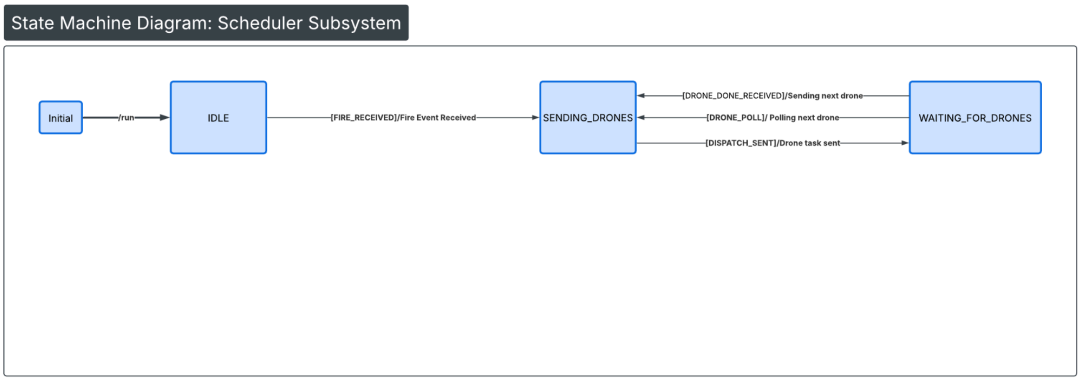
FAULTED (Recoverable Fault - DRONE_STUCK)
- Drone stuck mid-flight after configured delay
- Remains in FAULTED state for 3000ms recovery period
- Two possible transitions:
- → RETURNING if recovery successful (Event: RECOVERED)
- → OFFLINE if recovery fails

OFFLINE (Non-recoverable Fault - NOZZLE_JAM)
- Drone permanently disabled due to nozzle jam
- No transitions from this state
- Drone removed from service pool

Transitions:
- TASK_RECEIVED: Scheduler sends DRONE_TASK
- ARRIVED: Drone reaches zone
- FAULT_DETECTED: Mid-flight fault detected (DRONE_STUCK)
- HARD_FAULT: Nozzle jam detected at arrival (NOZZLE_JAM)
- DROP_COMPLETE: Agent deployment finished

- RETURN_COMPLETE: Base reached, auto-refill
- RECOVERED: Recovery period completed successfully

2.2.2 Scheduler State Machine:



The Scheduler manages three states coordinating drone assignments:

IDLE (Initial State)

- No pending fires, no busy drones
- Waiting for fire events from FireIncidentSubsystem
- Transition: → FIRE_RECEIVED when new fire event arrives

FIRE_RECEIVED (Fire Detected)

- New fire event added to pending queue
- Evaluates available drones using findBestDroneForNextEvent()
- Two possible transition based on drone availability:
  - → SENDING_DRONES if available drone with sufficient capacity found
  - → WAITING_FOR_DRONES if no suitable drone available
- Event: FIRE_RECEIVED

SENDING_DRONES (Dispatching)

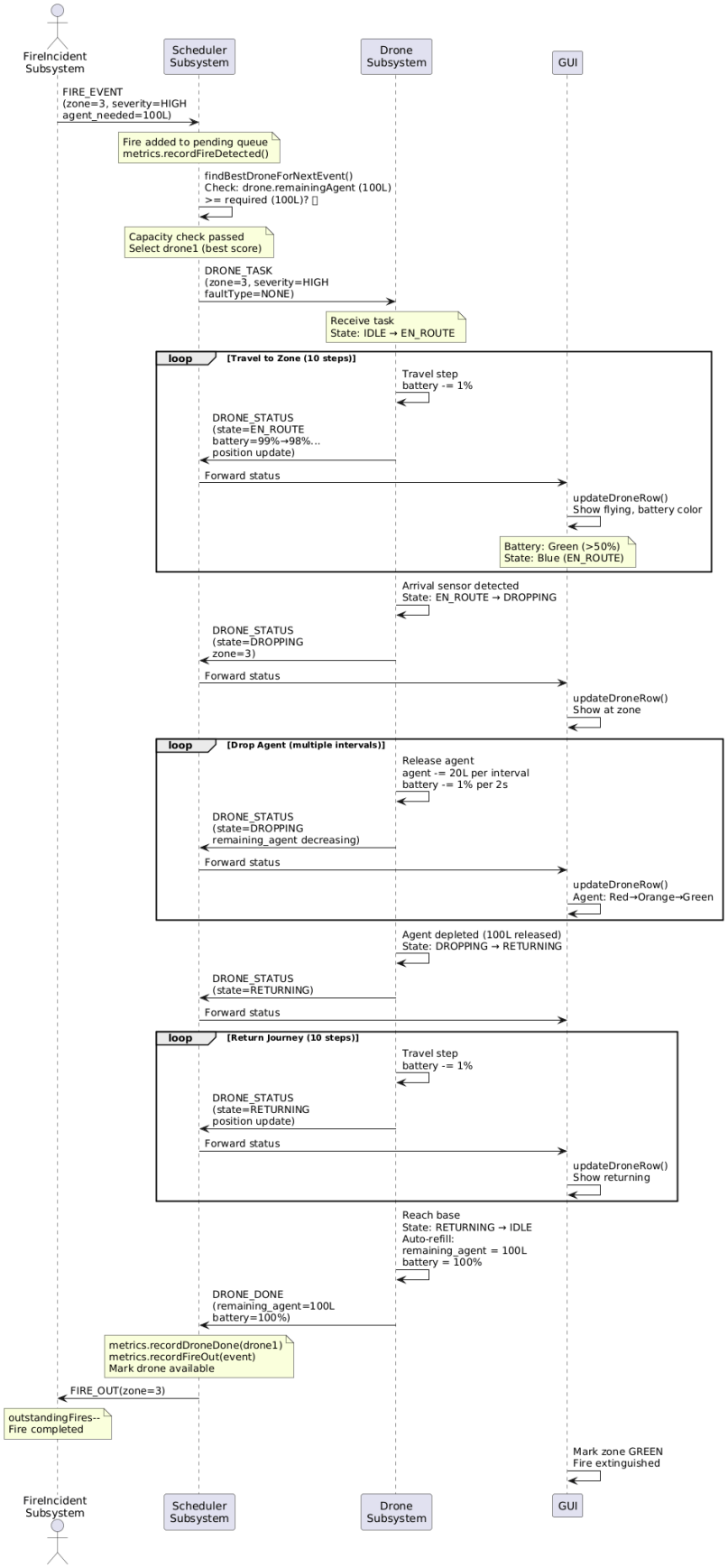
- Selecting and dispatching drone to fire zone
- Sends DRONE_TASK command with zone, severity, fault type
- Marks drone as busy
- Transitions back to WAITING_FOR_DRONES or FIRE_RECEIVED
- Event: DISPATCH_SENT

WAITING_FOR_DRONES (Queue Management)

- Pending fire waiting in queue
- All suitable drones are currently busy
- Can transition back to SENDING_DRONES when:
  - Drone completes task and becomes available (Event: DRONE_DONE_RECEIVED)
  - New drone available after fault recovery (Event: DRONE_STATUS_RECEIVED)
- Can transition to IDLE if all fires extinguished and no busy drones.

2.3 Sequence Diagrams

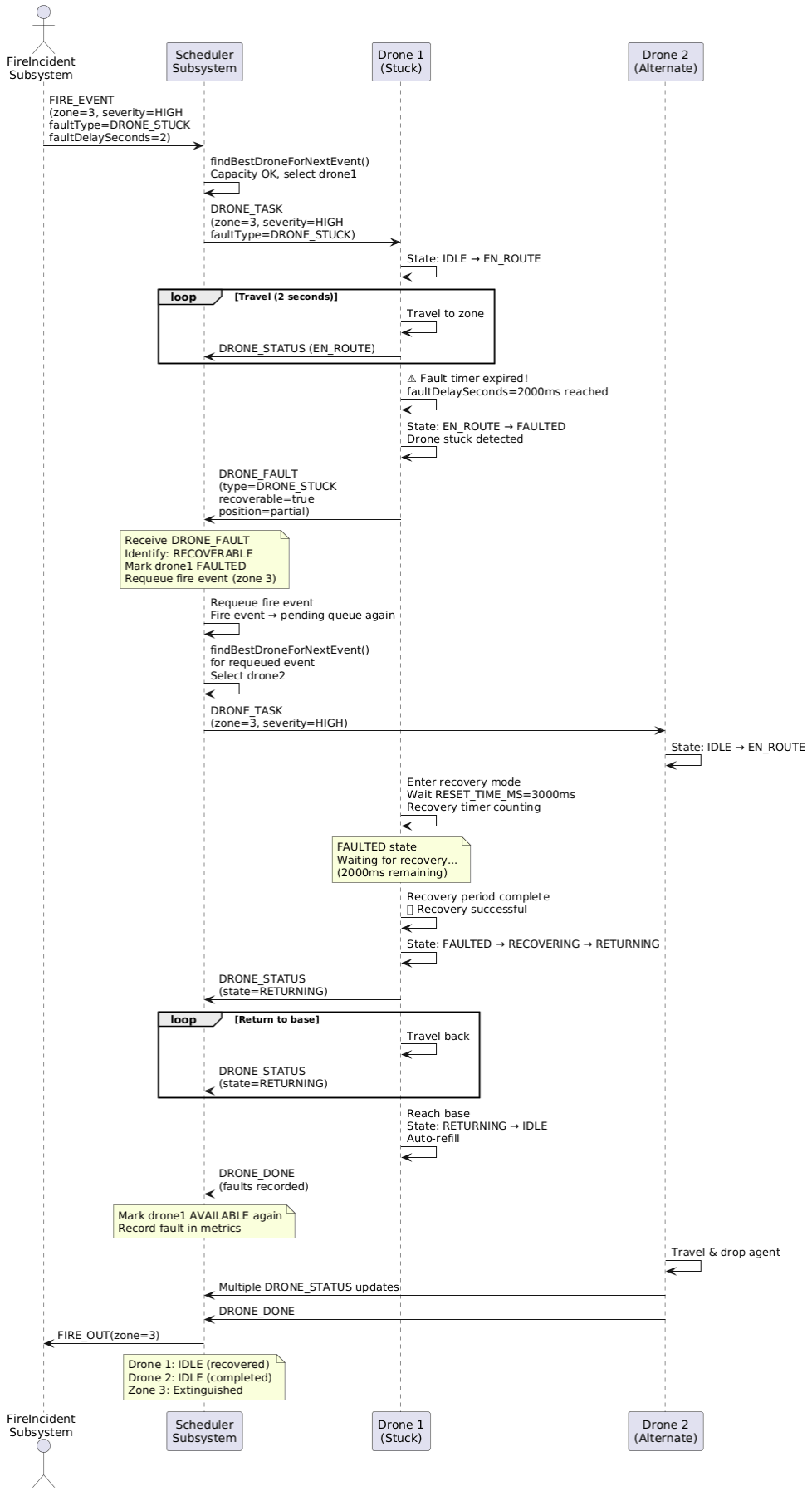
2.3.1. Sequence Diagram for Normal Fire Response



This sequence shows the complete flow when a fire is detected and successfully extinguished:

1. **FireIncidentSubsystem** sends FIRE_EVENT (zone=3, severity=HIGH, agent_needed=100L)
2. **SchedulerSubsystem** receives event, add to pending queue
3. **SchedulerSubsystem** calls findBestDroneForNextEvent(event):
   - Checks drone1.remainingAgent (100L) ≥ 100L required → Pass
   - Calculates score = distance(base→zone3) + missionsCompleted × 10
   - Selects best drone (assume drone1)
4. **SchedulerSubsystem** sends DRONE_TASK to **DroneSubsystem**
   - Message: zone=3, severity=HIGH, faultType=NONE
5. **DroneSubsystem** receives command:
   - Transitions: IDLE → EN_ROUTE
   - Starts travelToZone()
   - Each step: battery drain 1%, send DRONE_STATUS
6. **SchedulerSubsystem** receives DRONE_STATUS updates, forwards to **GUI**
   - **GUI** displays drone traveling with position updates
   - Indicator: Blue (EN_ROUTE) and Green (Battery > 50%)
7. **DroneSubsystem** arrives at zone:
   - Transitions: EN_ROUTE → DROPPING
   - Sends DRONE_STATUS with state=DROPPING
8. **DroneSubsystem** drops agent:
   - Releases 100L over multiple intervals
   - Agent depletes 20L per interval
   - Each 2s: battery drain 1%
   - Sends status updates
9. **SchedulerSubsystem** receives updates, forwards to **GUI**
   - **GUI** shows state=DROPPING, remaining_agent decreasing
10. **DroneSubsystem** completes drop:
    - Transitions: DROPPING → RETURNING
    - Starts travelToBase()
11. **DroneSubsystem** reaches base:
    - Transitions: RETURNING → IDLE
    - Auto-refill: remainingAgent=100L, batteryLevel=100%
    - Sends DRONE_DONE
12. **SchedulerSubsystem** receives DRONE_DONE:
    - Records metrics (distance, time, mission count)
    - Sends FIRE_OUT to **FireIncidentSubsystem**
    - Zone removed from active zone
13. **FireIncidentSubsystem** receives FIRE_OUT, decrements outstandingFires.
14. **SchedulerSubsystem** forwards all updates to **GUI**
    - **GUI** marks zone as extinguished (green background)
    - Drone returns to IDLE state

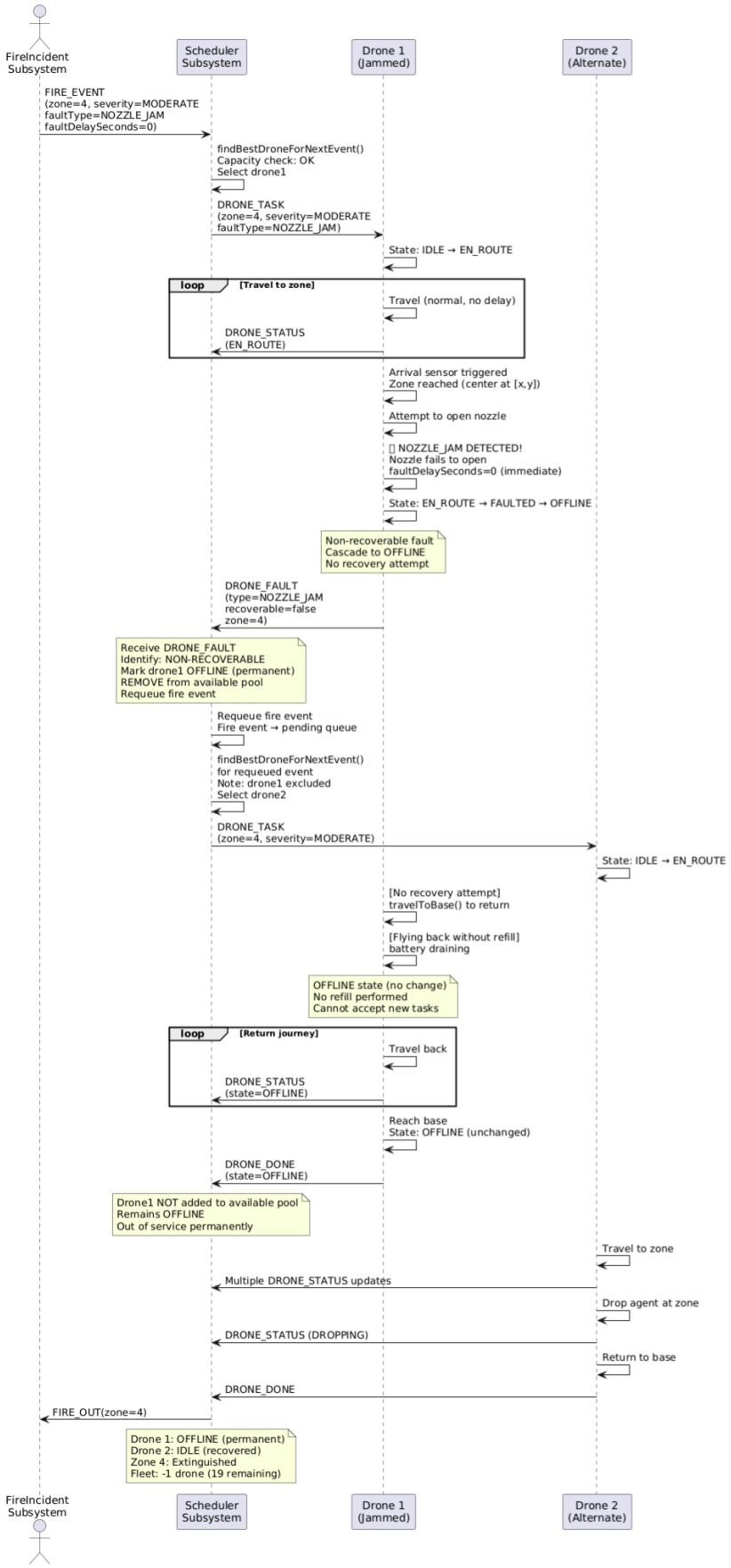
2.3.2 Sequence diagram for DRONE_STUCK



This sequence shows how the system detects and recovers from a DRONE_STUCK fault during flight:

1. **FireIncidentSubsystem** sends FIRE_EVENT with faultType=DRONE_STUCK, faultDelaySeconds=2
2. **SchedulerSubsystem** dispatches **DroneSubsystem** with DRONE_TASK including fault info
3. **DroneSubsystem** begins travelToZone() with injected fault:
   - Travels normally for 2 seconds
   - After faultDelaySeconds elapsed, detects DRONE_STUCK
4. **DroneSubsystem** fault handling:
   - Transitions: EN_ROUTE → FAULTED
   - Sends DRONE_FAULT message to **SchedulerSubsystem**
   - Records current position (partway to zone)
5. **SchedulerSubsystem** receives DRONE_FAULT:
   - Identifies as recoverable (DRONE_STUCK)
   - Marks drone as FAULTED (temporarily unavailable)
   - Requeues original fire event back to pending queue
   - Calls findBestDroneForNextEvent() to select alternate drone (if available)
6. **DroneSubsystem** enters recovery mode:
   - Waits 3000ms (RESET_TIME_MS)
   - Transitions: FAULTED → RECOVERING
7. **DroneSubsystem** recovery complete:
   - Transitions: RECOVERING → RETURNING
   - Begins travelToBaseFrom(currentX, currentY) to return to base
   - Continues draining battery during return
8. **DroneSubsystem** reaches base:
   - Transitions: RETURNING → IDLE
   - Auto-refill: remainingAgent=100L, batteryLevel=100%
   - Sends DRONE_DONE with reduced stats (fault recorded)
9. **SchedulerSubsystem** receives DRONE_DONE:
   - Records fault occurrence
   - Makes drone available again
10. **SchedulerSubsystem** detects requeued fire event in pending queue
11. **SchedulerSubsystem** calls findBestDroneForNextEvent() again:
    - May select same drone (now recovered) or different drone
    - Repeats dispatch sequence
12. **GUI** displays:
    - Drone color changes to RED when FAULTED
    - Fault indicator shown in drone status panel
    - When recovered, returns to normal color and state

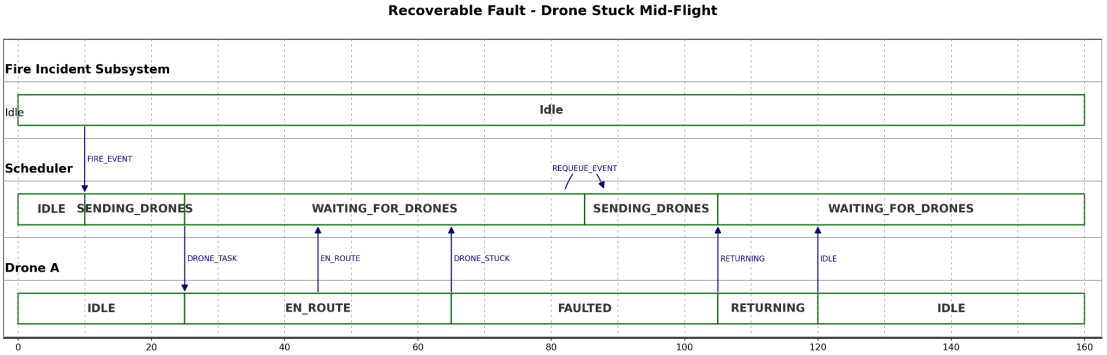
2.3.3 Sequence Diagram for NOZZLE_JAM

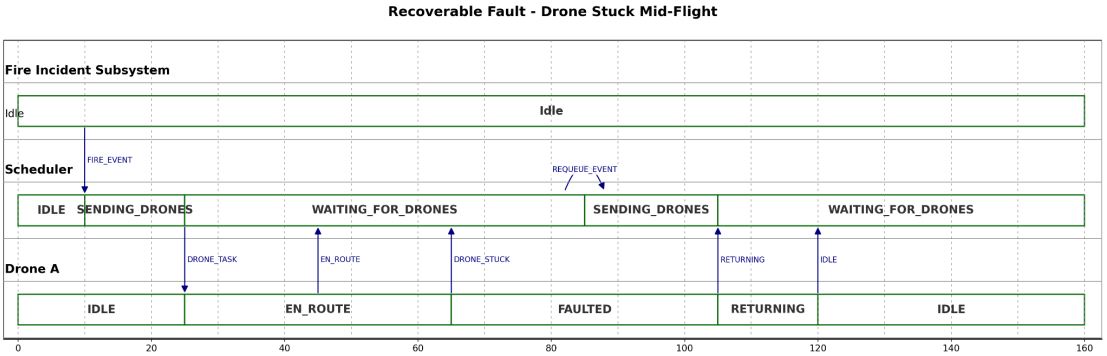


This sequence shows how the system handles a permanent NOZZLE_JAM fault:

1. **FireIncidentSubsystem** sends FIRE_EVENT with faultType=NOZZLE_JAM, faultDelaySeconds=0
2. **SchedulerSubsystem** dispatches **DroneSubsystem** with DRONE_TASK including fault info
3. **DroneSubsystem** begins travelToZone():
   - Travels normally (no delay for nozzle jam)
   - Reaches zone successfully
   - Transitions: EN_ROUTE → about to transition to DROPPING
4. **DroneSubsystem** detects NOZZLE_JAM at arrival:
   - Attempts to open nozzle
   - Detects jam condition
   - Transitions: EN_ROUTE → FAULTED → OFFLINE
   - Sends DRONE_FAULT (non-recoverable)
5. **SchedulerSubsystem** receives DRONE_FAULT:
   - Identifies as non-recoverable (NOZZLE_JAM)
   - Marks drone OFFLINE permanently
   - Removes from available drone pool
   - Requeues original fire event back to pending queue
6. **DroneSubsystem** after OFFLINE state:
   - Does NOT attempt recovery
   - Begins immediate travelToBase() to return for maintenance
   - No further fire responses (remains in OFFLINE state)
7. **DroneSubsystem** reaches base:
   - Sends DRONE_DONE with state=OFFLINE
   - Remains offline (no auto-refill, no task acceptance)
8. **SchedulerSubsystem** receives DRONE_DONE:
   - Records fault as permanent
   - Drone remains out of service
9. **SchedulerSubsystem** has requeued fire event and remaining available drones
10. **SchedulerSubsystem** calls findBestDroneForNextEvent() for requeued event:
    - Excludes offline drone from consideration
    - Selects from remaining available drones
    - If only 1 drone in system: event cannot be serviced
11. **GUI** displays:
    - Offline drone shown in BLACK color
    - OFFLINE status indicator
    - Fire zone may remain unserviced (shown in red/orange)
12. **FireIncidentSubsystem** does NOT receive FIRE_OUT
    - Zone remains in outstandingFires
    - Simulates real-world scenario of insufficient resources

2.4 Timing Diagrams:





These 2 timing diagrams show the temporal sequence of events when a DRONE_STUCK fault occurs mid-flight and the system recovers.

3. Team Responsibilities

Iteration 1:

Ma Chenwei:
- Code for Subsystems

Shah Asaad:
- Made UML Class diagram

Ye Edmund:
- Implemented test files

Zhang Derrick:
- Code for base classes and foundation for the message transporter (project setup and structure for subsystems)

Iteration 2:

Ma Chenwei:
- Implemented test files

Shah Asaad:
- Implemented drone state machine

Ye Edmund:
- Updated UML Class Diagrams by adding states and events
- Created state machine diagrams for both scheduler and drone subsystems

Zhang Derrick:
- Implemented SubSystems.GUI to have 4 grids, keeps track of active fires and if drone is busy or not (currently tracked through SystemCounts class)
- Implemented scheduler state machine

Iteration 3:

Ma Chenwei:
- Updated UML CLass Diagram
- Fixed few bugs

Shah Asaad:
- Implemented UDP for Fire incident and Drone subsystems
- Updated SubSystems.GUI

Ye Edmund:
- Implemented UDP communication for scheduler subsystem
- Added Drone logic and travel calculations using Iteration 0 values

Zhang Derrick:
- Added scheduling logic for drone selection: Dispatches drones that are reachable, pick drone with least amount of missions, tie break user drone with lower ID.
- Added SENDING_DRONE state to scheduler
- Added new set of test for new udp system

- Deprecated old tests

Iteration 4:

Ma Chenwei:
- Edit Class Diagram
- Fault type enumeration and made for support fault injection

Shah Asaad:
- Added the following tests:
- Unit tests for drone mission execution and state transitions
- Unit tests for scheduler queueing, dispatching, and reassignment logic
- Unit tests for fault injection handling
- Integration tests for main use case flow across all subsystems under normal and faulty conditions

Ye Edmund:
- Implemented dynamic drone scheduling
- Revamped GUI to show drone state and fault changes, and drone arrival to middle of zone
- Implemented Drone positioning and tracking

Zhang Derrick:
- Added backwards compatibility to some deprecated tests as to not break build
- Implemented faults to drones, drone_stuck being a short pause and return to base, and nozzle being an offline return to base unusable

Iteration 5:

Ma Chenwei:
- Updated Class Diagram for battery, scheduler metrics, GUI, etc.
- Fixed battery bugs
- Updated dynamic zones

Shah Asaad:
- Updated UI to show drone location, fire status, and faults
- Updated state diagrams

Ye Edmund:
- Updated UI to show drone location, fire status, and faults
- Updated state diagrams

Zhang Derrick:
- Added battery charge to drones
- Fixed stacking fire in zones bug
- Created timing diagrams for 2 fault situations

4. Setup and Test Instructions

4.1 Prerequisites

Java JDK 25
JUnit 5
An **IDE** such as **IntelliJ IDEA** or **Eclipse**

4.2 To Run the Project

Install **Java JDK 25**
Clone or **download** the **repository**
Open the project in your **IDE**
Edit the run configuration to use **Java 25** and set **Main.java** as the main class
Run the project

4.3 To Run Tests

Install **JUnit 5** dependencies in your IDE or build tool
Navigate to the test files in the src directory
Run **each test** individually or via your IDE's test runner

5. Measurement Results

```
==================================================
SIMULATION METRICS REPORT
==================================================
Simulation wall-clock runtime: 484.40 s
Time from first incident to last fire extinguished: 28418.82 s
Completed incidents: 50

Average Event Response Time: 37.95 s
Maximum Event Response Time: 46.68 s
Average Event Completion Time: 126.26 s
Maximum Event Completion Time: 143.10 s

Per-drone metrics:
Drone 1 | distance=44355.7 m | idle=28296.78 s | en_route=228.30
s | dropping=302.28 s | returning=235.92 s | faulted=0.00 s |
missions=6 | faults=0 | utilization=2.64%
Drone 2 | distance=36044.0 m | idle=28346.88 s | en_route=203.58
s | dropping=302.28 s | returning=210.54 s | faulted=0.00 s |
missions=6 | faults=0 | utilization=2.46%
Drone 3 | distance=36044.0 m | idle=28347.12 s | en_route=203.52
s | dropping=302.34 s | returning=210.30 s | faulted=0.00 s |
missions=6 | faults=0 | utilization=2.46%
Drone 4 | distance=46851.7 m | idle=28277.40 s | en_route=236.76
s | dropping=302.22 s | returning=246.90 s | faulted=0.00 s |
missions=6 | faults=0 | utilization=2.70%
Drone 5 | distance=45018.3 m | idle=28286.94 s | en_route=231.84
s | dropping=301.68 s | returning=242.82 s | faulted=0.00 s |
missions=6 | faults=0 | utilization=2.67%
Drone 6 | distance=35269.4 m | idle=28426.98 s | en_route=187.86
s | dropping=251.46 s | returning=196.98 s | faulted=0.00 s |
missions=5 | faults=0 | utilization=2.19%
Drone 7 | distance=41609.8 m | idle=28391.94 s | en_route=205.50
s | dropping=251.76 s | returning=214.08 s | faulted=0.00 s |
missions=5 | faults=0 | utilization=2.31%
Drone 8 | distance=42381.1 m | idle=28389.06 s | en_route=207.06
s | dropping=251.82 s | returning=215.34 s | faulted=0.00 s |
missions=5 | faults=0 | utilization=2.32%
Drone 9 | distance=34982.7 m | idle=28436.10 s | en_route=184.08
s | dropping=251.64 s | returning=191.46 s | faulted=0.00 s |
missions=5 | faults=0 | utilization=2.16%
Drone 10 | distance=34606.8 m | idle=28437.96 s | en_route=182.94
s | dropping=251.88 s | returning=190.50 s | faulted=0.00 s |
missions=5 | faults=0 | utilization=2.15%
Drone 11 | distance=37772.0 m | idle=28421.16 s | en_route=190.98
s | dropping=251.94 s | returning=199.20 s | faulted=0.00 s |
missions=5 | faults=0 | utilization=2.21%
Drone 12 | distance=44254.9 m | idle=28379.70 s | en_route=211.26
s | dropping=251.88 s | returning=220.44 s | faulted=0.00 s |
missions=5 | faults=0 | utilization=2.35%
Drone 13 | distance=37000.6 m | idle=28423.68 s | en_route=190.14
s | dropping=251.64 s | returning=197.82 s | faulted=0.00 s |
missions=5 | faults=0 | utilization=2.20%
Drone 14 | distance=36661.7 m | idle=28425.06 s | en_route=189.18
s | dropping=251.70 s | returning=197.34 s | faulted=0.00 s |
missions=5 | faults=0 | utilization=2.20%
Drone 15 | distance=41060.6 m | idle=28396.80 s | en_route=203.04
s | dropping=251.70 s | returning=211.74 s | faulted=0.00 s |
```

```
missions=5 | faults=0 | utilization=2.29%
Drone 16 | distance=37874.1 m | idle=28414.86 s | en_route=194.58
s | dropping=251.82 s | returning=202.02 s | faulted=0.00 s |
missions=5 | faults=0 | utilization=2.23%
Drone 17 | distance=29888.9 m | idle=28461.66 s | en_route=171.60
s | dropping=251.82 s | returning=178.20 s | faulted=0.00 s |
missions=5 | faults=0 | utilization=2.07%
Drone 18 | distance=28676.5 m | idle=28473.18 s | en_route=165.84
s | dropping=251.64 s | returning=172.62 s | faulted=0.00 s |
missions=5 | faults=0 | utilization=2.03%
Drone 19 | distance=26592.5 m | idle=28485.42 s | en_route=160.20
s | dropping=251.70 s | returning=165.96 s | faulted=0.00 s |
missions=5 | faults=0 | utilization=1.99%
Drone 20 | distance=27728.5 m | idle=28479.90 s | en_route=162.42
s | dropping=251.58 s | returning=169.38 s | faulted=0.00 s |
missions=5 | faults=0 | utilization=2.01%

Fleet averages:
Average drone idle time: 28399.93 s
Average drone en-route time: 195.53 s
Average drone dropping time: 264.34 s
Average drone returning time: 203.48 s
Average drone faulted time: 0.00 s
Total fleet distance travelled: 744673.8 m
==================================================
```

6. Design Reflection

Overall, the entire system runs smoothly as a complete integrated solution. The decoupled architecture with three independent subsystems communicating via UDP works effectively. FireIncidentSubsystem feeds fire events → Scheduler coordinates drones → DroneSubsystem executes missions → GUI displays real-time status. The state machines prevent invalid operations, capacity constraints are enforced, faults are handled gracefully, and metrics are collected automatically. The system demonstrates solid distributed systems design and handles multiple concurrent scenarios without conflicts.

However, there are several aspects of the system that could be improved. The graphical user interface (GUI), while functional, is relatively simple and lacks detailed visualization. Currently, the GUI only displays drone states (e.g., IDLE, EN_ROUTE, DROPPING), battery and agent levels as numerical values, and fire zones as colored grid cells. While this provides basic information, it does not offer an intuitive or user-friendly view of the system.

If given more time, we would enhance the GUI by adding real-time drone movement visualization on a map and clearer status indicators (such as icons or animations). For example, displaying drone paths, highlighting active missions, and using visual alerts for faults would significantly improve usability.

In conclusion, while the current system achieves its core functionality and demonstrates strong performance, further improvements in visualization and scheduling optimization would enhance both usability and overall system efficiency.